Simulando o Eee PC 1015BX no VirtualBox

Projeto DeX Station — guia de bancada para testar sem o netbook

Versão 1.0 · 03/11/2026 · Asus Eee PC 1015BX + Samsung Galaxy S24

> **Para que serve este guia.** O netbook físico está temporariamente indisponível. Enquanto ele não volta, a distro modificada (imagem Debian 13 + LXQt + o daemon `dex-station-session`) pode ser testada numa máquina virtual com parâmetros equivalentes aos do 1015BX. Este documento vai da instalação do VirtualBox até o roteiro de testes, e diz em cada etapa o que o resultado vale — e o que **não** vale.
>
> **Aviso que precisa ir para o artigo.** A VM reproduz o **comportamento**, não o **desempenho**. FPS, uso de CPU, temperatura, autonomia e tempo de boot medidos nela não substituem a medição no hardware real. A seção 13 detalha item por item o que pode e o que não pode ser citado como resultado.

Conteúdo

Seção	Assunto	Seção	Assunto
1	O que a VM resolve (e o que não resolve)	9	Deixando a VM parecida com o netbook
2	Requisitos do computador anfitrião	10	Roteiro de testes (T1 a T14)
3	Instalando o VirtualBox	11	Problemas comuns
4	Perfil de hardware equivalente ao 1015BX	12	Snapshots e rotina de bancada
5	Criando a máquina virtual	13	O que o método não prova
6	Instalando o sistema dentro da VM	14	Checklist de aceite da rodada
7	Configuração pós-instalação	15	Apêndices: script e referências
8	Conectando o Galaxy (o "celular" da VM)		

1. O que a VM resolve (e o que não resolve)

Item	Na VM	No 1015BX real	Consequência
Ligar e ver se a imagem dá boot	Sim, com UEFI	Sim	**Válido** para validar a imagem
Autologin, LXQt, daemon, scrcpy	Sim	Sim	**Válido** (funcional)
Detecção de celular por `adb`	Sim (rede ou USB)	Sim	**Válido** com ressalvas (§8)
Orçamento de RAM (1,5 GiB)	Sim, 1536 MB	Sim	**Válido** para "cabe ou não cabe"

Item	Na VM	No 1015BX real	Consequência
Desempenho (FPS, CPU, latência)	Otimista	Referência	**Não válido** como número final
Driver de vídeo `radeon` + UVD3	Não (é `vmwgfx`)	Sim	**Não reproduz** vídeo/consumo reais
Disco 5400 rpm de verdade	Aproximado	Sim	**Parcial** (ver §9)
Temperatura, bateria, autonomia	Não existe	Sim	**Não reproduzível** (T12)
USB 2.0 com limite de 500 mA	Só o barramento	Sim	**Não reproduzível**
Wi-Fi Atheros AR9285	Não (e1000)	Sim	Irrelevante para o projeto

Regra prática: **na VM se decide "funciona / não funciona"**. Os números de desempenho só entram no artigo depois de repetidos no netbook.

2. Requisitos do computador anfitrião

Requisito	Mínimo	Recomendado
Memória	8 GB	16 GB
Espaço livre	15 GB	40 GB (imagem + snapshots)
CPU	2 núcleos com VT-x/AMD-V	4 núcleos
Sistema	Linux, Windows 10/11 ou macOS	Linux (mesmo ambiente do build)
VirtualBox	7.0	7.1 ou mais novo
Extension Pack	Necessário só para USB 2.0	Instalado

Confira a virtualização antes de tudo:

```
# Linux: deve aparecer vmx (Intel) ou svm (AMD) e o módulo kvm carregado
grep -cE 'vmx|svm' /proc/cpuinfo
lsmod | grep -E '^kvm'

# VirtualBox presente?
VBoxManage --version
VBoxManage list extpacks
```

No Windows, o Hyper-V e o "Isolamento de núcleo" (Core Isolation) impedem o VirtualBox de usar a virtualização por hardware. Se `VBoxManage` reclamar de VT-x indisponível, desligue o Hyper-V em *Recursos do Windows* e reinicie.

3. Instalando o VirtualBox

3.1 Linux (Linux Mint, Ubuntu, Debian)

O caminho mais simples é o repositório da distribuição:

```
sudo apt update
sudo apt install virtualbox virtualbox-ext-pack
sudo usermod -aG vboxusers "$USER"   # necessário para USB
```

Depois de adicionar o grupo, **encerre a sessão e entre de novo** (ou reinicie), senão o passthrough de USB não funciona. Confira com:

```
groups | grep vboxusers
VBoxManage list extpacks        # lista o Extension Pack
```

Se a versão do repositório for antiga e faltar alguma opção deste guia, use o pacote oficial do site da Oracle (`virtualbox-7.1`), lembrando que ele substitui o pacote da distribuição.

3.2 Windows e macOS

1. Baixe o instalador e o **Extension Pack da mesma versão** em virtualbox.org.
2. Instale o VirtualBox, depois dê duplo clique no arquivo `.vbox-extpack` e aceite a licença.
3. No Windows, mantenha o Hyper-V desligado (ver §2).

3.3 Conferindo a instalação

```
VBoxManage --version
VBoxManage list ostypes | grep -i debian    # deve mostrar Debian (64-bit)
```

4. Perfil de hardware equivalente ao 1015BX

Estes são os parâmetros usados no script da seção 5. Eles foram escolhidos a partir dos dados medidos na bancada (ver `docs/ambiente-de-testes.md`).

Componente	1015BX real	VM	Por que assim
CPU	AMD C-60, 2 núcleos, 1,0 GHz	2 vCPU, limite de execução 70%	mantém o paralelismo e freia a CPU moderna
Memória	1,5 GiB úteis (2 GB menos vídeo)	**1536 MB**	força o mesmo orçamento de RAM
Vídeo	Radeon HD 6290 (`radeon`, UVD3)	VMSVGA, 128 MB, **3D desligado**	software rendering, o pior caso parecido
Disco	ST9320325AS, 320 GB, 5400 rpm	VDI 40 GB em SATA, cache do host desligado	espaço menor e I/O menos otimizado
Tela	1024×600, 10,1"	modo 1024×600 forçado no guest	alvo do display virtual
Firmware	UEFI + ESP (GPT)	**EFI**	a imagem é GPT com `EFI/BOOT/BOOTX64.EFI`
USB	3× USB 2.0 (480 Mbps)	controlador EHCI	mesma largura de banda
Rede	AR9285 + Ethernet 10/100	bridged, e1000	a VM fica na mesma rede do celular

Componente	1015BX real	VM	Por que assim
Áudio	HDA	desligado	áudio é opcional e fica off no projeto
Bateria / térmica	86% de saúde / sensores	não existem	não reproduzível (T12)

Dois pontos que costumam confundir:

- **1536 MB não é pouco por acidente.** É o valor medido no netbook (`MemTotal` ≈ 1,5 GiB). Se a distro não couber nisso na VM, também não caberia no netbook.
- **3D desligado é proposital.** O `radeon` do 1015BX não tem o mesmo caminho de aceleração do `vmwgfx`. Deixar o 3D ligado cria uma máquina mais rápida e menos parecida com o alvo.

5. Criando a máquina virtual

5.1 Pela interface gráfica

1. **Novo** → nome `dex-station`, tipo *Linux*, versão *Debian (64-bit)*.
2. Memória **1536 MB**; processadores **2**; marque *Habilitar EFI*.
3. Disco: *Criar um disco virtual agora* → VDI, **40 GB**, alocação dinâmica.
4. **Configurações → Sistema → Placa-mãe**: chipset PIIX3, EFI marcado; **Processador**: *Habilitar VT-x/AMD-V* e *PAE/NX* marcados, *Virtualização aninhada* desmarcada.
5. **Tela**: controlador **VMSVGA**, memória de vídeo **128 MB**, *Aceleração 3D* desmarcada.
6. **Armazenamento**: controlador SATA (AHCI); em *Opções*, desmarque *Usar cache de I/O do host*. Se for instalar do zero, adicione a ISO num controlador IDE.
7. **Rede**: *Placa em modo bridge*, interface = a do seu computador (Wi-Fi ou cabo).
8. **USB**: controlador **USB 2.0 (EHCI)**.
9. **Áudio**: desmarcado.

5.2 Por linha de comando (recomendado)

O repositório traz `scripts/vm-virtualbox.sh`, que faz tudo isso de forma repetível. O uso é:

```
# a partir da raiz do repositório (o caminho tem espaços: use aspas)
cd "/caminho/para/dex-station"
chmod +x scripts/vm-virtualbox.sh

# opção A: testar a imagem já gerada pelo build
./scripts/vm-virtualbox.sh imagem build/dex-station.img

# opção B: começar de um disco vazio e instalar Debian do zero
./scripts/vm-virtualbox.sh vazio
./scripts/vm-virtualbox.sh iso ~/isos/debian-13.0.0-amd64-netinst.iso

# ligar, desligar, ver configuração

./scripts/vm-virtualbox.sh iniciar
./scripts/vm-virtualbox.sh parar
./scripts/vm-virtualbox.sh info
```

O script aceita variáveis de ambiente para ajustar o perfil:

```
VM=dex-teste RAM=2048 CPUS=2 CAP=100 \
  ./scripts/vm-virtualbox.sh imagem build/dex-station.img
```

`CAP` é o limite de execução da CPU (em %). 70% é um bom ponto de partida para aproximar a velocidade do C-60; com 100% a VM fica rápida demais para qualquer comparação.

5.3 O que o script faz por dentro

Se preferir entender (ou reproduzir na mão), a sequência é esta:

```
VM=dex-station

VBoxManage createvm --name "$VM" --ostype Debian_64 --register
VBoxManage modifyvm "$VM" \
  --cpus 2 --memory 1536 --vram 128 --ioapic on \
  --firmware efi --graphicscontroller vmsvga --accelerate3d off \
  --cpuexecutioncap 70 --nested-hw-virt off --paravirtprovider none \
  --mouse usbtablet --keyboard ps2 \
  --audio-enabled off --usb-ehci on --usb-ohci on \
  --boot1 disk --boot2 dvd --boot3 none --boot4 none

DEV=$(ip route | awk '/default/{print $5; exit}')

VBoxManage modifyvm "$VM" --nic1 bridged --bridgeadapter1 "$DEV"

VBoxManage storagectl "$VM" --name SATA --add sata \
  --controller IntelAhci --portcount 2 --hostiocache off

# disco vazio de 40 GB (só na opção B)
VBoxManage createmedium disk --filename ~/VMs/dex-station.vdi \
  --size 40960 --variant Standard
VBoxManage storageattach "$VM" --storagectl SATA --port 0 --device 0 \
  --type hdd --medium ~/VMs/dex-station.vdi
```

Para a opção A, o disco não é criado: a imagem crua é **convertida** em VDI.

```
VBoxManage convertfromraw build/dex-station.img \
  ~/VMs/dex-station.vdi --format VDI
```

> **Cuidado:** não use `VBoxManage modifymedium --resize` numa imagem que já tem tabela GPT feita por *loop device*. O redimensionamento deixa o cabeçalho de backup do GPT fora do lugar e o GRUB passa a não encontrar a ESP. Se precisar mesmo crescer o disco, faça a correção **dentro** do guest com `sudo sgdisk -e /dev/sda` e confira com `lsblk`.

6. Instalando o sistema dentro da VM

6.1 Rota A — dar boot na imagem do projeto (recomendada para testar)

Esta é a rota que valida o artefato de verdade: a mesma imagem que iria para o pendrive ou para o HD.

1. Crie a VM com `./scripts/vm-virtualbox.sh imagem build/dex-station.img`.
2. Inicie. A VM entra no **GRUB** direto, porque a imagem tem `EFI/BOOT/BOOTX64.EFI` gravado com `grub-install --removable`.
3. Se aparecer o **shell do EFI** em vez do GRUB, digite:

```
Shell> FS0:
FS0:\> \EFI\BOOT\BOOTX64.EFI
```

Isso confirma que o arquivo está no lugar e só faltou a entrada de boot — o mesmo teste vale para o pendrive no netbook.

4. No GRUB, edite a linha do kernel (`e`) e **apague** `quiet` na primeira rodada. Você precisa ver as mensagens de inicialização para diagnosticar tela preta e falha de montagem.
5. Login: usuário `dex` (autologin no LXQt). Se o autologin não subir, entre no TTY com `chvt 2` (a partir de um terminal, ou pelo SSH — ver §11).

6.2 Rota B — instalar Debian 13 do zero (para desenvolver o build)

Use esta rota quando o objetivo for mexer no `post-install.sh` ou em pacotes, e não testar a imagem final.

1. `./scripts/vm-virtualbox.sh vazio` e depois `./scripts/vm-virtualbox.sh iso <debian.iso>`.
2. No instalador: idioma *Português (Brasil)*, teclado *ABNT2*.
3. Particionamento **manual**, seguindo o desenho do projeto:

Partição	Tamanho	Sistema de arquivos	Ponto de montagem
`sda1` (ESP)	512 MB	`fat32`	`/boot/efi` (flag `boot`)
`sda2` (raiz)	restante do disco	`ext4`	`/`
swap	—	—	**não criar**: o projeto usa zram

4. Usuário `dex`, senha simples de bancada; não instale *task* de desktop no `tasksel`.
5. Ao terminar, entre na VM e rode o script do projeto:

```
sudo apt update
sudo apt install -y git
# copie o repositório para a VM (ssh/scp) e execute:
sudo ./build/post-install.sh
```

6. Troque a ordem de boot para o disco: `./scripts/vm-virtualbox.sh boot disco`.

6.3 Rota C — modo Live (teste T8, "HD intocado")

O teste T8 exige provar que o pendrive não altera o disco interno. Na VM isso é feito com **dois discos**:

```
# disco "HD" (o que não pode mudar) + imagem do projeto como disco de boot
./scripts/vm-virtualbox.sh vazio            # cria o disco de 40 GB (sda)
# anexe a imagem convertida como segundo disco
VBoxManage convertfromraw build/dex-station.img ~/VMs/dex-live.vdi --format VDI
VBoxManage storageattach dex-station --storagectl SATA --port 1 --device 0 \
  --type hdd --medium ~/VMs/dex-live.vdi
```

Antes de ligar, tire os checksums do disco "HD":

```
sha256sum ~/VMs/dex-station.vdi > /tmp/antes.sha256
```

Depois do teste, desligue a VM e compare:

```
sha256sum -c /tmp/antes.sha256
```

O `OK` prova que a imagem Live não escreveu no disco — o mesmo argumento do netbook real, com a vantagem de o checksum ser verificável sem desmontar nada.

7. Configuração pós-instalação

7.1 Tela em 1024×600

Sem os *Guest Additions*, a VM não redimensiona sozinha. Descubra o nome da saída e force o modo:

```
xrandr | head -20          # no VMSVGA a saída chama-se Virtual-1
cvt 1024 600 60            # gera a modeline
```

Copie a modeline devolvida pelo `cvt` e aplique (o número pode variar levemente):

```
xrandr --newmode "1024x600_60.00" 38.25 1024 1048 1112 1184 \
  600 601 604 620 -HSync +Vsync
xrandr --addmode Virtual-1 "1024x600_60.00"
xrandr --output Virtual-1 --mode "1024x600_60.00"
```

Para persistir, crie `~/.config/autostart/tela-1024x600.desktop` chamando um script com esses três comandos. **Não** use `CustomVideoMode1` (extradata do VirtualBox): ele depende do BIOS legado e não funciona com EFI.

7.2 Teclado ABNT2

```
sudo localectl set-keymap br-abnt2
setxkbmap -layout br -variant abnt2     # vale para a sessão atual
```

Teste acentuação e cedilha no DeX, não só no terminal da VM: o caminho real é `teclado físico → adb → Android`, e é lá que o layout pode divergir.

7.3 Rede

```
ip -4 addr show          # a VM bridged deve ter IP da mesma faixa do celular
ping -c1 1.1.1.1
```

Se a VM estiver em NAT (quando não há interface bridge), o acesso à internet funciona, mas o celular na rede local **não** será alcançável por `adb connect` — nesse caso use a opção de passthrough USB (§8.2).

7.4 Relógio

```
timedatectl               # no Debian/systemd
sudo timedatectl set-ntp true
```

O relógio da VM acompanha o do anfitrião, então ele é confiável — ao contrário do netbook, que está sem NTP. Só não copie a data da VM para o documento sem conferir o fuso (`America/Recife`, UTC−3).

7.5 Serviço do daemon

Dependendo da fase do projeto, o daemon roda como serviço ou como autostart:

```
systemctl status dex-station       # imagem Debian
sv status dex-station              # se estiver testando no Void
dex-station-launch status          # CLI do próprio projeto
journalctl -u dex-station -f       # acompanhar em tempo real
```

7.6 Guest Additions: não instale na imagem de teste

Os *Guest Additions* trazem mouse integrado, redimensionamento automático e pastas compartilhadas — e, junto com eles, drivers e serviços que **não existem no 1015BX**. Isso contamina justamente as medidas de RAM e de CPU.

Use-os apenas numa **cópia** da VM, para conforto de desenvolvimento. A imagem que vai para o netbook não deve tê-los.

8. Conectando o Galaxy (o "celular" da VM)

Sem o celular, o daemon só mostra a tela de espera. Há três caminhos, em ordem de preferência.

8.1 `adb` por TCP/IP (recomendado)

Não depende de passthrough de USB e funciona igual em qualquer anfitrião.

1. No celular: *Configurações → Opções do desenvolvedor → Depuração sem fio* → ative.
2. Toque em *Parear dispositivo com código de pareamento* e anote **IP:porta** e o **código**.
3. Na VM:

```
sudo apt install -y android-tools-adb     # se a imagem ainda não tiver adb
adb pair 192.168.1.50:37123               # IP:porta da tela de pareamento
# digite o código de 6 dígitos quando pedir

adb connect 192.168.1.50:5555             # porta mostrada na tela principal
adb devices
adb shell getprop ro.product.model        # deve responder SM-S921B
```

4. Confirme que o daemon viu o aparelho:

```
journalctl -u dex-station -n 30
```

> **A pegadinha:** a porta de pareamento muda a cada vez e o Android desliga a depuração sem fio quando o Wi-Fi cai ou o aparelho reinicia. Se `adb devices` mostrar `offline`, rode `adb kill-server` e pareie de novo.

Se o aparelho for Android 10 ou anterior, o caminho é o antigo `adb tcpip 5555`, que exige **um** cabo USB antes:

```
adb devices          # com o cabo, autorize no celular
adb tcpip 5555
adb connect 192.168.1.50:5555
```

8.2 Passthrough de USB (quando quiser testar plug/unplug de verdade)

Pré-requisitos: Extension Pack instalado, usuário no grupo `vboxusers` e controlador **USB 2.0 (EHCI)** na VM.

```
VBoxManage list usbhost | grep -A4 -i samsung   # descubra VendorId/ProductId
VBoxManage usbfilter add 0 --target dex-station \
  --name "Galaxy S24" --vendorid 04e8 --productid 6860
```

Na janela da VM, use *Dispositivos → USB → Samsung Galaxy* para conectar. Dentro do guest, o `adb` precisa da regra udev do Android (o pacote `android-tools-adb` já a instala).

Limitações conhecidas: o dispositivo cai quando a VM reinicia, e a reconexão às vezes exige desconectar e reconectar o cabo no anfitrião. O modo §8.1 é mais estável para o dia a dia.

8.3 Sem celular: `adb` falso

Boa parte do daemon pode ser testada sem hardware nenhum, com um `adb` de mentira à frente do `PATH` — é o que o projeto já prevê em `tests/test-state-machine.sh`:

```
mkdir -p /tmp/fakebin
cat > /tmp/fakebin/adb <<'EOF'
#!/bin/sh
case "$1" in
  devices) echo "List of devices attached"; echo "ABC123    device" ;;
  *) exit 0 ;;
esac
EOF
chmod +x /tmp/fakebin/adb
PATH=/tmp/fakebin:$PATH dex-station-launch start
```

Assim você valida a máquina de estados, o hotplug lógico e as mensagens de erro sem depender do celular — e sem gastar a bateria dele.

9. Deixando a VM parecida com o netbook (e onde isso falha)

Recurso	Como aproximar	Limite
CPU lenta	`--cpuexecutioncap 70` (ou 50)	o C-60 não tem AVX; a VM expõe AVX2 e pode escolher outro caminho de código
Disco lento	`--hostiocache off` e VDI no HDD, não no SSD	o *seek* do SSD do anfitrião continua existindo
Disco lento (fino)	grupo de banda (abaixo)	nem toda versão aceita limite em disco
RAM apertada	1536 MB fixos	a VM tem overhead próprio do hipervisor
Sem GPU	VMSVGA + 3D off	o `vmwgfx` não é o `radeon`, e não há UVD3
Sem bateria	nada a fazer	`BAT0` não existe no guest; o daemon precisa tratar isso
Sem térmica	nada a fazer	não há sensor no guest

Limite de banda de disco (opcional; confira a sintaxe na sua versão):

```
VBoxManage bandwidthctl dex-station add LimiteDisco --limit 25
VBoxManage storageattach dex-station --storagectl SATA --port 0 --device 0 \
  --type hdd --medium ~/VMs/dex-station.vdi --bandwidthgroup LimiteDisco
```

> **Teste que a VM faz melhor que o netbook:** "e quando não existe bateria?". No guest, `BAT0` simplesmente não aparece. Se o daemon quebrar com isso, é bug — e você descobriu de graça, antes de o netbook voltar.

10. Roteiro de testes (T1 a T14)

A matriz completa está no `PLAN.md`. A tabela abaixo diz como executar cada teste na VM e o quanto o resultado vale.

Teste	Como executar na VM	Vale como resultado?
T1 — chroot coerente	`journalctl -b -p err`	Sim
T2 — boot completo	cronômetro + `systemd-analyze`	Parcial: a VM é mais rápida
T3 — detecção do celular	`adb devices` e log do daemon	Sim

Teste	Como executar na VM	Vale como resultado?
T4 — DeX virtual	`scrcpy --new-display=1024x600/160 --max-fps=15`	Sim (funcional, não FPS)
T5 — entrada e ABNT2	digitar acentos/ç dentro do DeX	Sim
T6 — hotplug	`adb disconnect` e `adb connect`	Parcial: o unplug físico é outro caminho
T7 — fallback A→B→C	forçar `METHOD=overlay` e observar	Sim
T8 — Live USB	rota C com dois discos + checksums	Sim
T9 — rede pelo USB	`adb shell ping -c1 1.1.1.1`	Sim
T10 — instalação no HD	recriar o VDI do zero e repetir o script	Sim
T11 — performance	`free -m`, `top -b -n1`, 30 min	**Não**: números otimistas
T12 — energia	—	**Não reproduzível**
T13 — recuperação	`sudo chvt 2`, `dex-station restart`	Sim
T14 — autonomia	30 min sem celular	Parcial (sem bateria)

Comandos úteis para as medições que **são** válidas:

```
free -m                                 # cabe no orçamento de 1,5 GiB?
ps -eo rss,comm --sort=-rss | head -12  # quem está comendo a RAM
systemd-analyze blame | head -10        # o que atrasa o boot
journalctl -u dex-station -b --no-pager   # o que o daemon fez nesta sessão
```

Para T6 sem cabo, simule a desconexão pelo `adb` — é o mesmo evento que o daemon enxerga:

```
adb disconnect                                  # "despluga"
adb connect 192.168.1.50:5555                   # "plugou de novo"
```

Para T13, o atalho de TTY do teclado costuma ser capturado pelo anfitrião. O caminho confiável é pedir a troca de console de dentro do guest:

```
sudo chvt 2      # vai para o TTY 2
sudo chvt 7      # volta para o gráfico
```

11. Problemas comuns

Sintoma	Causa provável	Solução
"FATAL: No bootable medium found"	imagem sem ESP ou ordem de boot errada	no shell EFI, `FS0:` e `\EFI\BOOT\BOOTX64.EFI`; confira `--boot1 disk`
Cai no shell do EFI sempre	falta `EFI/BOOT/BOOTX64.EFI` na imagem	refaça o `grub-install --removable` na imagem
Tela preta depois do GRUB	driver/controlador de vídeo	troque VMSVGA ↔ VBoxSVGA, mantenha 3D off; teste `nomodeset`

Sintoma	Causa provável	Solução
Resolução presa em 800×600	sem Guest Additions e sem modeline	§7.1
Mouse "preso" ou sem integração	dispositivo apontador	`VBoxManage modifyvm dex-station --mouse usbtablet`
`adb devices` vazio	rede da VM em NAT, celular em outra faixa	use bridge (§5.1) ou USB (§8.2)
`adb` diz `offline`	depuração sem fio caiu	`adb kill-server`, parear de novo
USB do celular não aparece	Extension Pack ou grupo	instale o extpack, `usermod -aG vboxusers`, relogue
VM lenta demais	sem VT-x ou limite de CPU baixo	confira §2; suba `CAP`
Erro "VT-x is not available"	Hyper-V/KVM conflitando	desligue Hyper-V (Windows) ou libere o KVM
Relógio errado no guest	fuso ou NTP	`timedatectl set-timezone America/Recife`
Áudio mudo / estourado	áudio desligado na VM	espere isso: áudio é opcional no projeto

Para diagnosticar boot sem interface, use a janela em modo *headless* e tire uma captura:

```
VBoxManage startvm dex-station --type headless
VBoxManage controlvm dex-station screenshotpng /tmp/tela.png
```

12. Snapshots e rotina de bancada

Antes de cada teste que possa sujar a instalação, tire um snapshot. É o equivalente ao "backup antes de mexer" do netbook, mas instantâneo.

```
./scripts/vm-virtualbox.sh snapshot antes-do-t4
./scripts/vm-virtualbox.sh restaurar antes-do-t4
```

Rotina sugerida por rodada:

1. `restaurar base-limpa` (ou o snapshot da imagem recém-convertida).
2. Ligar, esperar a tela de espera, anotar hora e consumo de RAM.
3. Executar o teste da rodada (T3, T4, T6...).
4. Guardar evidências: `journalctl` da sessão, `screenshotpng`, `free -m`.
5. Desligar e restaurar o snapshot.

Evidências que valem anexar no relatório:

```
journalctl -b --no-pager > /tmp/t4-journal.txt
VBoxManage showvminfo dex-station > /tmp/t4-vminfo.txt
```

13. O que o método não prova

1. **Desempenho.** A CPU da VM é muito mais rápida por núcleo e tem AVX; o C-60 não tem. FPS, latência e uso de CPU medidos aqui são **piso**, não resultado.
2. **Vídeo.** O guest usa `vmwgfx`/`vboxvideo`, não o `radeon`. O scrcpy vai decodificar por software nos dois casos, mas o custo por quadro não é comparável.
3. **Disco.** Mesmo com cache desligado e limite de banda, o anfitrião tem SSD e o 1015BX tem um HDD de 5400 rpm com 320 GB.
4. **Energia.** Não há bateria, nem consumo de 500 mA na porta USB, nem throttling térmico. T12 continua exigindo o netbook.
5. **Wi-Fi.** A VM usa `e1000`; o netbook usa `ath9k`. Nada do projeto depende disso, mas não cite comportamento de rede do netbook com base na VM.
6. **Firmware.** O EFI do VirtualBox não é o do 1015BX. O que se valida é "a imagem tem ESP e boota por `BOOTX64.EFI`", não o comportamento do firmware Asus.

Frase para o artigo: *"Os testes funcionais foram executados em máquina virtual com configuração equivalente (2 vCPU, 1,5 GiB, disco SATA); os números de desempenho e energia foram medidos apenas no Asus Eee PC 1015BX."*

14. Checklist de aceite da rodada

- ☐ VirtualBox e Extension Pack na mesma versão; usuário no grupo `vboxusers`
- ☐ VM com 2 vCPU, 1536 MB, VMSVGA 128 MB, 3D desligado, EFI
- ☐ Disco SATA com cache do host desligado
- ☐ Boot pelo `BOOTX64.EFI` (rota A) ou instalação concluída (rota B)
- ☐ Tela em 1024×600, teclado ABNT2, relógio com NTP
- ☐ `adb devices` mostrando o S24 (ou `adb` falso, no teste de estado)
- ☐ Tela de espera do daemon aparecendo
- ☐ `scrcpy --new-display` abrindo o DeX
- ☐ Snapshot tirado antes e depois do teste
- ☐ Evidências salvas (`journalctl`, `screenshotpng`, `free -m`)
- ☐ Limitações da seção 13 citadas no relatório

Apêndice A — script completo

O arquivo `scripts/vm-virtualbox.sh` do repositório implementa tudo o que foi descrito.

```
#!/usr/bin/env bash
# vm-virtualbox.sh — cria e opera a VM de bancada do DeX Station (Eee PC 1015BX)
set -euo pipefail

VM="${VM:-dex-station}"
RAM="${RAM:-1536}"
CPUS="${CPUS:-2}"
CAP="${CAP:-70}"
VDI_DIR="${VDI_DIR:-$HOME/VMs}"
DISK_MB="${DISK_MB:-40960}"
VDI="$VDI_DIR/$VM.vdi"
```

```
erro() { echo "erro: $*" >&2; exit 1; }

need_vbox() {
  command -v VBoxManage >/dev/null 2>&1 || erro "VBoxManage não encontrado"
}

iface_padrao() {
  ip -o -4 route show default 2>/dev/null | awk '{print $5; exit}'
}

existe() {
  VBoxManage list vms 2>/dev/null | grep -q "\"$VM\""

}

criar_base() {
  need_vbox
  if existe; then
    erro "a VM '$VM' já existe; rode '$0 remover' antes de recriar"
  fi
  local bridge
  bridge="$(iface_padrao || true)"
  VBoxManage createvm --name "$VM" --ostype Debian_64 --register
  VBoxManage modifyvm "$VM" \
    --cpus "$CPUS" --memory "$RAM" --vram 128 --ioapic on \

   --firmware efi --graphicscontroller vmsvga --accelerate3d off \
    --cpuexecutioncap "$CAP" --nested-hw-virt off --paravirtprovider none \
    --mouse usbtablet --keyboard ps2 \
    --audio-enabled off --usb-ehci on --usb-ohci on \
    --boot1 disk --boot2 dvd --boot3 none --boot4 none
  if [ -n "$bridge" ]; then
    VBoxManage modifyvm "$VM" \
      --nic1 bridged --bridgeadapter1 "$bridge" --nictype1 82540EM
  else
    VBoxManage modifyvm "$VM" --nic1 nat --natpf1 "ssh,tcp,,2222,,22"
  fi
  VBoxManage storagectl "$VM" --name SATA --add sata \

   --controller IntelAhci --portcount 2 --hostiocache off
  VBoxManage storagectl "$VM" --name IDE --add ide
  mkdir -p "$VDI_DIR"
  echo "VM '$VM' criada (RAM ${RAM} MB, ${CPUS} vCPU, limite ${CAP}%)"
}

cmd_vazio() {
  criar_base
  VBoxManage createmedium disk --filename "$VDI" \
    --size "$DISK_MB" --variant Standard
  VBoxManage storageattach "$VM" --storagectl SATA --port 0 --device 0 \
    --type hdd --medium "$VDI"

 echo "disco vazio de ${DISK_MB} MB em $VDI"
  echo "próximo passo: $0 iso <debian-13-netinst.iso>"
}

cmd_imagem() {
  local img="${1:-}"
  [ -n "$img" ] || erro "uso: $0 imagem <arquivo.img>"
  [ -f "$img" ] || erro "imagem não encontrada: $img"
  criar_base
  echo "convertendo $img em VDI (o .img não é alterado)…"
  VBoxManage convertfromraw "$img" "$VDI" --format VDI
  VBoxManage storageattach "$VM" --storagectl SATA --port 0 --device 0 \

   --type hdd --medium "$VDI"
  VBoxManage snapshot "$VM" take "imagem-limpa" >/dev/null
  echo "pronto: $0 iniciar"
}

cmd_iso() {
  local iso="${1:-}"
  [ -n "$iso" ] || erro "uso: $0 iso <arquivo.iso>"
  [ -f "$iso" ] || erro "ISO não encontrada: $iso"
  VBoxManage storageattach "$VM" --storagectl IDE --port 1 --device 0 \
    --type dvddrive --medium "$iso"
  VBoxManage modifyvm "$VM" --boot1 dvd --boot2 disk
```

```
 echo "ISO anexada; boot pela unidade de DVD"
}

cmd_boot() {
  case "${1:-}" in
    disco) VBoxManage modifyvm "$VM" --boot1 disk --boot2 dvd ;;
    dvd)   VBoxManage modifyvm "$VM" --boot1 dvd --boot2 disk ;;
    *)     erro "uso: $0 boot disco|dvd" ;;
  esac
  echo "ordem de boot: ${1}"
}


cmd_iniciar()  { need_vbox; VBoxManage startvm "$VM" --type "${1:-gui}"; }
cmd_parar()    { VBoxManage controlvm "$VM" acpipowerbutton; }
cmd_snapshot() {
  VBoxManage snapshot "$VM" take "${1:?uso: $0 snapshot <nome>}"
}
cmd_restaurar() {
  VBoxManage snapshot "$VM" restore "${1:?uso: $0 restaurar <nome>}"
}
cmd_info()     { VBoxManage showvminfo "$VM" | sed -n '1,45p'; }
cmd_remover()  { VBoxManage unregistervm "$VM" --delete; echo "removida"; }

case "${1:-}" in

 vazio)     shift; cmd_vazio "$@" ;;
  imagem)    shift; cmd_imagem "$@" ;;
  iso)       shift; cmd_iso "$@" ;;
  boot)      shift; cmd_boot "$@" ;;
  iniciar)   shift; cmd_iniciar "$@" ;;
  parar)     shift; cmd_parar ;;
  snapshot)  shift; cmd_snapshot "$@" ;;
  restaurar) shift; cmd_restaurar "$@" ;;
  info)      shift; cmd_info ;;
  remover)   shift; cmd_remover ;;
  *)         sed -n '2,22p' "$0" | sed 's/^# \{0,1\}//'; exit 1 ;;
esac
```

Apêndice B — comandos úteis

```
# --- VirtualBox ---
VBoxManage list vms                       # VMs registradas
VBoxManage showvminfo dex-station         # configuração completa
VBoxManage snapshot dex-station list      # snapshots existentes
VBoxManage controlvm dex-station screenshotpng /tmp/tela.png
VBoxManage controlvm dex-station usbattach <uuid>   # plugar o celular
VBoxManage list usbhost                   # dispositivos disponíveis no host

# --- guest (imagem Debian) ---
xrandr                                    # saídas e modos
adb devices && adb shell getprop ro.product.model
scrcpy --new-display=1024x600/160 --max-fps=15 --no-audio

free -m && ps -eo rss,comm --sort=-rss | head
journalctl -u dex-station -b --no-pager

# --- evidências ---
journalctl -b --no-pager > /tmp/journal.txt
VBoxManage showvminfo dex-station > /tmp/vminfo.txt
```

Apêndice C — referências

- VirtualBox — manual do usuário e `VBoxManage`: https://www.virtualbox.org/manual/ · https://www.virtualbox.org/manual/ch08.html
- Debian 13 "trixie" — guia de instalação: https://www.debian.org/releases/trixie/
- scrcpy — display virtual e opções: https://github.com/Genymobile/scrcpy
- Android — depuração sem fio: https://developer.android.com/tools/adb
- Projeto DeX Station — `PLAN.md` (§8, §9.1.1, §11), `docs/ambiente-de-testes.md`